

Codetantra Practical 2

Practice Lab Assignment

21.1. List operations

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- Add: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- Remove: Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- Display: Displays the current list of integers. If the list is empty, display "List is empty".
- Quit: Exits the program.

The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Sample Test Cases

Test case 1

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 10

List after adding: [10]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 3

Integer: 10

List after adding: [10, 20]

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 1

Integer: 30

Element not found

1. Add

2. Remove

3. Display

4. Quit

Enter choice: 3

Integer: 30

intOps.py

```
1 def list_operations():
2     my_list = []
3     while True:
4         print("1. Add\n2. Remove\n3. Display\n4. Quit")
5         choice = input("Enter choice: ")
6
7         if choice == "1":
8             try:
9                 val = int(input("Integer: "))
10                my_list.append(val)
11                print("List after adding: ", my_list)
12            except ValueError:
13                print("Invalid input")
14
15        elif choice == "2":
16            if not my_list:
17                print("List is empty")
18            else:
19                try:
20                    val = int(input("Integer: "))
21                    if val in my_list:
22                        my_list.remove(val)
23                        print("List after removing: ", my_list)
24                    else:
25                        print("Element not found")
26                except ValueError:
27                    print("Invalid input")
28
29        elif choice == "3":
30            if not my_list:
31                print("List is empty")
32            else:
33                print(my_list)
34
35        elif choice == "4":
36            break
37
38        else:
39            print("Invalid choice")
40
41        if __name__ == "__main__":
42            list_operations()
```

Execution time: 0.143 s | 0.321 s | 102.0 ms

2 out of 3 shown test cases passed

1 out of 1 hidden test cases passed

Test case 1

Expected output:

```
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 10
List after adding: [10]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 3
Integer: 10
List after adding: [10, 20]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 30
Element not found
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 3
Integer: 30
```

Actual output:

```
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 10
List after adding: [10]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 3
Integer: 10
List after adding: [10, 20]
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 30
Element not found
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 3
Integer: 30
```

Test case	Input	Output
Original Dictionary: (1: 'A', 2: 'B', 3: 'C', 4: 'D', 5: 'E', 6: 'F', 7: 'G', 8: 'H', 9: 'I', 10: 'J', 11: 'K', 12: 'L', 13: 'M', 14: 'N', 15: 'O', 16: 'P', 17: 'Q', 18: 'R', 19: 'S', 20: 'T', 21: 'U', 22: 'V', 23: 'W', 24: 'X', 25: 'Y', 26: 'Z')	Actual output Original Dictionary: (1: 'A', 2: 'B', 3: 'C', 4: 'D', 5: 'E', 6: 'F', 7: 'G', 8: 'H', 9: 'I', 10: 'J', 11: 'K', 12: 'L', 13: 'M', 14: 'N', 15: 'O', 16: 'P', 17: 'Q', 18: 'R', 19: 'S', 20: 'T', 21: 'U', 22: 'V', 23: 'W', 24: 'X', 25: 'Y', 26: 'Z')	
after insertion: (1: 'A', 2: 'B', 3: 'C', 4: 'D', 5: 'E', 6: 'F', 7: 'G', 8: 'H', 9: 'I', 10: 'J', 11: 'K', 12: 'L', 13: 'M', 14: 'N', 15: 'O', 16: 'P', 17: 'Q', 18: 'R', 19: 'S', 20: 'T', 21: 'U', 22: 'V', 23: 'W', 24: 'X', 25: 'Y', 26: 'Z')	Actual output after insertion: (1: 'A', 2: 'B', 3: 'C', 4: 'D', 5: 'E', 6: 'F', 7: 'G', 8: 'H', 9: 'I', 10: 'J', 11: 'K', 12: 'L', 13: 'M', 14: 'N', 15: 'O', 16: 'P', 17: 'Q', 18: 'R', 19: 'S', 20: 'T', 21: 'U', 22: 'V', 23: 'W', 24: 'X', 25: 'Y', 26: 'Z')	

Lab Assignment

The screenshot displays the CodeTANTRA web application interface. The top navigation bar includes a 'Course' tab, a search bar, and a 'Ask Gemini' button. The main content area is titled 'Essentials of Data Science Laboratory - 2304102L - 2026'. On the left, a sidebar lists various lab assignments, with '2.2.1 Linear search Technique' selected. The main panel shows the details of this lab, including the problem statement, input/output format, and sample test cases. The problem statement asks to write a program to check if a given element is present in an array using linear search. The input format specifies that the first line contains an array of integers and the second line contains the key element. The output format requires printing the index if found, or 'Not found' if not. The sample test case shows an input array of [1, 2, 3, 4, 5, 6] and a key of 3, resulting in an output of 2. The right panel shows the code editor with a Python implementation of the linear search algorithm. The code uses a try-except block to handle the input and output, and a for loop to iterate through the array. The bottom right panel shows the test results, indicating that 2 out of 2 shown test cases passed, with an average time of 0.013 s and a maximum time of 0.018 s. The test cases show the expected output [1, 2, 3, 4, 5, 6] and the actual output [1, 2, 3, 4, 5, 6] for the first case, and 2 for the second case.

Course

mitaoe.codetantra.com/secure/course.jsp?euclid=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d62/698c54860aba96214c8b7d64/64ac0cb9b4547106899a4e1b

CODETANTRA Home Learn Anywhere

202501090056@mitaoe.ac.in Support Logout

Essentials of Data Science Laboratory - 2304102L - 2026

Search course (ctrl)+k

1. Practical 1

2. Practical 2

2.1. Practice Lab Assignment

2.1.1. List operations

2.1.2. Dictionary Operations

2.2. Lab Assignment

2.2.1. Linear search Technique

2.2.2. Captain of the Team

3. Practical 3

4. Practical 4

5. Practical 5

2.2.1. Linear search Technique

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

Output format:

- If the element is found, print the index. If the element found multiple times, print the starting index
- If the element is not found, print **Not found**.

Sample Test Case:

Input:
1 2 3 4 5 6
3

Output:
2

Sample Test Cases

Test case 1

1 2 3 4 5 6

3

2

Test case 2

1 2 3 4 5 6 7 8 9 10

20

Not found

CTP1703...

```
1 try:
2     arr=list(map(int,input().split()))
3     key=int(input())
4     found=False
5     for i in range(len(arr)):
6         if arr[i]==key:
7             print(i)
8             found=True
9             break
10    if not found:
11        print("Not found")
12 except EOFError:
13    pass
```

Average time: 0.013 s, Maximum time: 0.018 s

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 (13 ms)

Expected output: 1 2 3 4 5 6

Actual output: 1 2 3 4 5 6

3

2

Test case 2 (10 ms)

Terminal Test cases

Prev Next Submit Next 2

Course

mitaoe.codetantra.com/secure/course.jsp?eucld=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d62/698c54860aba96214c8b7d64/6774d0abf4ab787eca9d9230

Ask Gemini

CODETANTRA

Home Learn Anywhere

202501090050@mitaoe.ac.in Support Logout

Essentials of Data Science
Laboratory - 2304102L - 2026

Search course (ctrl + k)

1. Practical 1

2. Practical 2

2.1. Practice Lab Assignment

2.1.1. List operations

2.1.2. Dictionary Operations

2.2. Lab Assignment

2.2.1. Linear search Technique

2.2.2. Captain of the Team

3. Practical 3

4. Practical 4

5. Practical 5

2.2.2. Captain of the Team

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:

The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format:

The output should be the height (in centimeters) of the tallest player.

Sample Test Cases

Test case 1

471 369 385 356 374 393 386 390 387 372 368

393

captainof...

```
1 try:
2     heights=list(map(int,input().split()))
3     if heights:
4         print(max(heights))
5 except EOFError:
6     pass
```

Average time 0.009 s 8.87 ms

Maximum time 0.013 s 13.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 43 ms

Expected output 471 369 385 356 374 393 386 390 387 372 368

Actual output 471 369 385 356 374 393 386 390 387 372 368

393

Terminal

Test cases

Prev

Reset

Submit

Next